

PRIVATEFIN

A Privacy-First Explainable Financial Advisory System

Project Idea 2: Financial Advisor Bot

CM3020 Artificial Intelligence — Draft Project Report

Table of Contents

Chapter 1: Introduction.....	5
1.1 Project Concept.....	5
1.2 Motivation.....	5
1.3 Research Methodology.....	6
1.4 Aims and Objectives.....	6
1.5 Scope and Limitations.....	6
1.6 Report Structure.....	7
Chapter 2: Literature Review.....	7
2.1 Introduction.....	7
2.2 Domain-Specific Large Language Models.....	7
2.2.1 BloombergGPT.....	7
2.2.2 FinGPT.....	8
2.3 Specialised NLP: FinBERT.....	8
2.4 Quantitative Data and Technical Indicators.....	9
2.4.1 yfinance.....	9
2.4.2 RSI and MACD.....	9
2.5 Regulatory and Ethical Landscape.....	10
2.5.1 EU AI Act (2024): Explainability as a Design Benchmark.....	10
2.5.2 GDPR: Data Sovereignty Architecture.....	10
2.6 Financial Forecasting and Machine Learning Alternatives.....	10
2.7 Explainable AI and LLM Risks in Finance.....	11
2.8 Synthesis and Research Gap.....	11
2.9 Methodology Justification.....	13
Chapter 3: Design.....	13
3.1 Design Objectives.....	13
3.1.1 Functional Requirements.....	13

3.1.2 Non-Functional Requirements.....	14
3.2 User Personas and Needs.....	14
3.3 System Architecture.....	15
3.4 User-Facing Interface Design.....	16
3.5 Component Design.....	19
3.5.1 Data Ingestion Layer.....	19
3.5.2 Indicator Engine Layer.....	19
3.5.3 OIIR Prompt Builder Layer.....	20
3.5.4 Local LLM Layer.....	20
3.5.5 Interface Layer.....	20
3.6 Technology Decisions.....	20
3.7 Methodology and Design Rationale.....	21
3.8 Evaluation Design.....	21
3.9 Work Plan and Contingency.....	22
Chapter 4: Implementation.....	23
4.1 Development Environment.....	23
4.2 Project Structure.....	23
4.3 Data Ingestion.....	24
4.4 Indicator Engine.....	24
4.5 OIIR Prompt Builder.....	25
4.6 Local LLM Integration.....	26
4.7 Streamlit User Interface.....	26
4.8 Unit Testing.....	27
4.9 Error Handling and Edge Cases.....	29
4.10 Implementation Gaps.....	29
Chapter 5: Evaluation.....	30
5.1 Evaluation Strategy.....	30

5.2 Component 1: Pipeline Correctness.....	30
5.3 Component 2: Privacy Architecture Audit.....	31
5.4 Component 3: OIIR Explainability Quality.....	32
5.5 Component 4: User Study.....	33
5.6 Critical Self-Assessment.....	34
5.7 Threats to Validity.....	35
5.8 Rubric Gaps Summary.....	35
Chapter 6: Conclusion.....	36
6.1 Summary.....	36
6.2 Objectives Scorecard.....	36
6.3 Originality.....	36
6.4 Future Work.....	37
6.5 Closing Reflection.....	37
References.....	38

Chapter 1: Introduction

1.1 Project Concept

PrivateFin is a locally executed, explainable AI financial advisory application designed for general and novice retail investors. The system combines deterministic technical indicator computation with Meta's Llama 3 large language model, served entirely on-device via Ollama. Its defining architectural property is Zero Data Egress: during LLM inference, no personal financial query data is transmitted to external servers. Every recommendation is accompanied by a structured reasoning trace following the Observe → Interpret → Infer → Recommend (OIIR) scaffold, ensuring outputs are auditable, explainable, and consistent with the transparency standard articulated in Article 13 of the EU AI Act (2024) — adopted here as a design benchmark (see 2.5.1) — and with the data protection principles of GDPR Articles 5 and 25.

This project is developed under **Template: CM3020 Artificial Intelligence — Project Idea 2: Financial Advisor Bot**, as specified in the University of London project guidelines. The template requires an AI system capable of providing structured financial recommendations from market data inputs. PrivateFin extends this template by integrating a locally served LLM as the reasoning layer, replacing the cloud-based inference pipelines typical of comparable systems, and by incorporating regulatory principles as first-class design constraints.

1.2 Motivation

The retail financial technology landscape presents a compounding set of problems for everyday investors. High-performance AI advisory systems — most notably BloombergGPT (Wu et al., 2023) and FinGPT (Yang et al., 2023) — route inference through cloud APIs, meaning every query implicitly discloses which securities a user is researching, and provider terms rarely preclude commercial use of interaction data for model improvement.

Second, recommendations from existing platforms are often opaque. Platforms such as eToro, Nutmeg, and robo-advisors like Betterment and Wealthify employ black-box portfolio optimisation models that produce Buy / Hold / Sell signals without any derivable audit trail. The EU AI Act (Regulation (EU) 2024/1689) attaches its most demanding transparency obligations to high-risk AI systems. Retail investment advisory is not itself among the use cases enumerated in Annex III — the finance-related entries concern creditworthiness assessment and risk pricing in life and health insurance — but the Act's Article 13 standard, that outputs be sufficiently transparent for users to interpret them appropriately, is adopted in this project as a deliberate design benchmark for consumer-facing financial AI. A system that states only “generated by a neural network” would not satisfy that standard.

Third, access to high-quality financial AI is gatekept by cost. Bloomberg Terminal subscriptions exceed £20,000 per annum, structurally excluding the retail investor

segment these tools nominally serve. Among the systems reviewed in Chapter 2, no open-source, fully local advisory tool with structured explainability was identified at consumer hardware price points.

PrivateFin is designed to close this **Transparency–Privacy Gap** through four integrated properties: fully local LLM inference; deterministic technical indicator computation providing a verifiable audit trail; structured OIIR prompting producing regulator-grade reasoning traces; and a lightweight Streamlit interface deployable on consumer hardware without cloud subscription.

1.3 Research Methodology

The project’s research method is a **hybrid symbolic–generative pipeline**: closed-form technical indicators (RSI, MACD) supply auditable quantitative grounding, and a local LLM supplies human-readable interpretation and cautious recommendations. This methodology was chosen deliberately over the reinforcement-learning, evolutionary-strategy, and backtesting approaches suggested by the CM3020 Financial Advisor Bot template brief; 2.6 and 2.9 justify this choice in detail once the relevant comparator systems and literature have been introduced.

The hybrid approach aligns the research method with the project’s three testable claims: privacy (architectural), determinism (computational), and explainability (linguistic and regulatory).

1.4 Aims and Objectives

PrivateFin is designed around three core claims, each testable and mapped to system architecture:

- **Privacy:** No personal financial query data is transmitted to external servers during LLM inference.
- **Determinism:** Quantitative indicator outputs are correct, closed-form calculations independently verifiable from public market data.
- **Explainability:** Reasoning constitutes meaningful transparency in the sense of the EU AI Act Article 13 benchmark adopted in 2.5.1 — structured, data-linked, and novice-accessible. The implementation deliverable is a web-based Streamlit advisor bot usable by non-technical users: company selection from 108 S&P 500-style equities, date range configuration, a risk-profile selector (Low, Medium, High) passed into the LLM prompt as context, live metric cards, price history chart, and structured OIIR narrative analysis — with educational disclaimers and graceful degradation when the local model is unavailable. No command-line interaction or coding is required; the user launches the application in a browser and receives analysis in plain English.

1.5 Scope and Limitations

PrivateFin currently analyses single equities (including dual-class share blending, e.g. GOOG/GOOGL). It is intended for educational and demonstration purposes, not

professional financial advice. At draft-report stage, several planned evaluation components (Wireshark privacy audit, TA-Lib cross-validation, formal user study) remain incomplete; Chapter 5 documents these honestly. Features described in early documentation but not yet implemented (prompt inspection pane, time-horizon control, extended indicator set, SQLite audit trail) are noted as future work in Chapters 4 and 6.

1.6 Report Structure

Chapter 2 surveys academic and technical foundations, critically evaluates prior financial AI systems, and justifies the chosen methodology. Chapter 3 presents system design, user-interface decisions, and the evaluation plan with precise parameters. Chapter 4 describes implementation of the full pipeline to date. Chapter 5 reports initial evaluation results and critical self-assessment. Chapter 6 summarises achievements, limitations, and future work.

Chapter 2: Literature Review

2.1 Introduction

The application of Large Language Models to retail finance has exposed a Transparency–Privacy Gap: high-capability AI advisory systems are simultaneously opaque, privacy-invasive, and inaccessible. This review surveys foundations across six areas, in order: domain-specific LLM architectures (Section 2.2), specialised NLP methodology (Section 2.3), quantitative data frameworks (Section 2.4), the regulatory landscape (Section 2.5), financial forecasting alternatives (Section 2.6), and explainable AI and LLM risks (Section 2.7). Section 2.8 synthesises the research gap; Section 2.9 justifies PrivateFin’s methodology.

2.2 Domain-Specific Large Language Models

2.2.1 BloombergGPT

BloombergGPT (Wu et al., 2023) is a 50-billion parameter decoder-only model trained on FinPile — a 363-billion token hybrid corpus comprising 51% proprietary Bloomberg data (earnings transcripts, analyst reports, regulatory filings) and 49% general internet corpora. Its distinguishing contribution is domain-weighted corpus construction rather than architectural novelty. On five domain-specific benchmarks — including Financial PhraseBank sentiment, FiQA question answering, and financial named entity recognition — BloombergGPT consistently outperforms GPT-NeoX-20B and OPT-66B despite comparable or smaller parameter counts, validating domain-weighted training as a more efficient approach than raw scale for financial tasks (Wu et al., 2023).

Its limitations are structural and directly motivate PrivateFin’s design. Deployment is proprietary SaaS via Bloomberg Terminal (exceeding £20,000 per annum), creating severe access gatekeeping for retail investors. Cloud-side inference transmits financial intent data to external servers; under GDPR Article 4(1), such query data constitutes personal data where reasonably linkable to a natural person (GDPR, 2016). Bloomberg’s terms do not

preclude commercial use of interaction data for model improvement. Finally, BloombergGPT produces no structured reasoning output: recommendations are generated autoregressively without enforced logical scaffolding — black-box behaviour that would fall short of the Article 13 transparency standard this project adopts as its benchmark. PrivateFin addresses all three failures: local execution eliminates data transmission; OIIR prompting mandates auditable reasoning traces.

2.2.2 FinGPT

FinGPT (Yang et al., 2023) adopts Low-Rank Adaptation (LoRA; Hu et al., 2022) for parameter-efficient fine-tuning. LoRA decomposes weight update matrices into two low-rank matrices, reducing trainable parameters by over 99% relative to full fine-tuning, making domain adaptation feasible without Bloomberg-scale compute budgets. The pipeline ingests real-time data from SEC EDGAR, Twitter/X, Reddit, and news APIs, fine-tuning foundation models (LLaMA, Falcon, ChatGLM) on financial instruction-following objectives. LoRA-fine-tuned LLaMA-7B variants achieve competitive performance against much larger general-purpose models, validating targeted adaptation over raw scale — a principle PrivateFin adopts through structured prompt engineering rather than fine-tuning.

FinGPT’s practical deployment, however, reconstructs the very cloud dependency it claims to avoid: documented implementations route inference through OpenAI GPT-3.5/4 APIs or Hugging Face Inference endpoints. Data sovereignty is therefore not achieved. Furthermore, FinGPT ingests speculative social media discourse alongside structured SEC filings without provenance weighting, making recommendations unanchored to verifiable market data. When a user asks “why this stock?”, FinGPT cannot produce a metric-traceable answer. PrivateFin resolves this by grounding all LLM reasoning in outputs from a deterministic quantitative layer, ensuring every justification references auditable, reproducible calculations.

2.3 Specialised NLP: FinBERT

FinBERT (Araci, 2019) applies domain-adaptive pre-training to BERT (Devlin et al., 2019), an encoder-only transformer architecture. Unlike autoregressive decoders, BERT’s bidirectional self-attention conditions each token’s representation on all surrounding tokens simultaneously, producing contextualised embeddings well-suited to discriminative classification. Araci (2019) initialises from BERT-base-uncased weights and continues pre-training on a financial corpus comprising Reuters TRC2 articles, earnings transcripts, and analyst reports, adapting internal representations to financial vocabulary (EBITDA, basis points, P/E ratios) and the hedged, forward-looking language that general sentiment lexicons misclassify. A softmax classification head fine-tuned on the Financial PhraseBank (Malo et al., 2014) reaches approximately 97% accuracy on the subset of the dataset on which all annotators agree, and approximately 86% on the full dataset — in both cases markedly above VADER and vanilla BERT — demonstrating that domain-specific pre-training yields disproportionate gains even at small model sizes.

FinBERT establishes the foundational principle that domain-targeted training outperforms general-purpose models even with fewer parameters — a principle PrivateFin extends to prompt engineering: saturating the LLM’s context with structured financial indicator data produces more accurate qualitative interpretations than vague natural language descriptions. However, FinBERT is architecturally incapable of generative tasks: it outputs a probability distribution over three sentiment classes, which does not constitute an explanation in any regulatory sense. Its typical inference pathway also relies on Hugging Face cloud checkpoints. PrivateFin synthesises FinBERT’s domain-specificity insight with the generative capability of a decoder-only LLM, while retaining full local execution.

2.4 Quantitative Data and Technical Indicators

2.4.1 yfinance

The yfinance library (Aroussi, 2019) retrieves OHLCV (Open, High, Low, Close, Volume) time-series data from Yahoo Finance as Pandas DataFrames (McKinney, 2010). Critically for a privacy-preserving system, yfinance requires no API key registration, eliminating the creation of user-linked data trails on third-party servers — a property absent in commercial alternatives such as Alpha Vantage, Bloomberg API, and Quandl. Its determinism is equally important: given identical parameters, yfinance returns identical data, enabling any historical recommendation to be independently audited. This reproducibility is foundational to PrivateFin’s traceability objective: the quantitative inputs to every LLM prompt can be independently verified from public market data.

2.4.2 RSI and MACD

Two indicators, computed deterministically via Pandas, serve as the primary structured inputs to PrivateFin’s reasoning layer. The Relative Strength Index (RSI; Wilder, 1978) is defined as $RSI = 100 - [100 / (1 + RS)]$, where RS is the ratio of average gain to average loss over a rolling 14-period window. RSI values in $[0, 30]$ signal oversold conditions; values in $[70, 100]$ signal overbought conditions. The MACD (Appel, 1979) is computed as $EMA(12) - EMA(26)$, with a signal line $EMA(9)$ of the MACD series; a bullish crossover occurs when MACD crosses above its signal line.

Both indicators are closed-form and deterministic: given identical OHLCV input, outputs are always identical. In PrivateFin, these values anchor the OIIR prompt, providing objective, verifiable data points that the LLM must reference in its reasoning chain, and constitute the audit trail underpinning the Article 13 transparency benchmark. Chain-of-Thought prompting (Wei et al., 2022) further structures this reasoning: by instructing the model to enumerate relevant indicators, their values, the thresholds evaluated, and the logical inference connecting data to recommendation, PrivateFin produces outputs that are both human-readable and benchmark-compliant — while the system prompt requests only the structured OIIR sections shown to the user, rather than any free-form reasoning preamble.

2.5 Regulatory and Ethical Landscape

2.5.1 EU AI Act (2024): Explainability as a Design Benchmark

The EU AI Act (Regulation (EU) 2024/1689) attaches its strictest obligations to High-Risk systems as defined by Article 6 and the enumerated use cases of Annex III. Retail investment advisory is not itself an Annex III category: the finance-adjacent entries cover creditworthiness assessment and credit scoring of natural persons, and risk assessment and pricing in life and health insurance. PrivateFin therefore does not claim to be a regulated high-risk system. Instead, it adopts the Act's Chapter III obligations voluntarily as design benchmarks, on the argument that AI-generated financial guidance for novice investors raises directly analogous transparency concerns: Article 13 (sufficient transparency for users to interpret outputs appropriately), Article 11 with Annex IV (technical documentation enabling reconstruction of the system's development and behaviour), and Article 14 (human oversight). Explainability under the Article 13 standard must be meaningful rather than nominal — the system must articulate the specific features and reasoning steps behind each output — which PrivateFin's structured OIIR prompting is designed to satisfy directly.

2.5.2 GDPR: Data Sovereignty Architecture

Three GDPR provisions shape PrivateFin's architecture. Article 22 grants a right not to be subject to a decision based *solely* on automated processing that produces legal or similarly significant effects. An advisory output that a human user reads and acts upon at their own discretion arguably falls outside this strict scope; nevertheless, the provision signals the level of safeguard expected wherever automation influences financially consequential choices, and PrivateFin adopts corresponding safeguards — educational disclaimers, cautious recommendations, and a human-readable rationale — by design. Articles 5 and 25 (data minimisation and Data Protection by Design) require protection principles to be embedded in the design rather than applied post hoc. Cloud inference sits uneasily with Article 25, since it necessitates transmitting financial query data to external processors; PrivateFin's Ollama-based local inference satisfies the principle by architectural construction.

2.6 Financial Forecasting and Machine Learning Alternatives

Neural time-series forecasters and reinforcement learning (RL) trading agents represent dominant research directions in computational finance, and are the techniques the CM3020 Financial Advisor Bot brief explicitly recommends. Concretely, an RL formulation of this project would model the advisor bot as a Markov Decision Process (or, more realistically given partial market observability, a POMDP) over a small action space — buy, hold, sell — with state derived from OHLCV history and indicator values, and reward defined by realised or simulated portfolio return. Evolutionary strategies would instead evolve a population of candidate trading-rule parameterisations against historical fitness (e.g. cumulative return or Sharpe ratio) rather than learning a value function directly.

Both formulations are methodologically mismatched to this project’s objectives. A trained RL policy network or evolved rule-set produces a decision (or action-value estimate) without an inspectable chain from market data to recommendation: the “reasoning,” such as it is, is encoded in learned weights or an evolved parameter vector, neither of which can be read back as a step-by-step justification meeting the Article 13 transparency benchmark. Both also typically require substantial historical training data and repeated retraining as market regimes shift, which conflicts with the deterministic, on-demand computation this project’s privacy and traceability claims depend on.

López de Prado (2018) demonstrates that backtest-driven strategy development is additionally vulnerable to data snooping, overfitting, and selection bias, producing impressive in-sample Sharpe ratios that fail out-of-sample. This is a general critique of backtesting as a validation method, and it applies most directly to systems that are themselves generating trading signals. PrivateFin’s deliverable is different in kind: it is an advisory system whose output is a human-facing explanation, not an executed trade or a discrete signal, so profitability metrics like Sharpe ratio would validate the wrong property even if backtesting were implemented without López de Prado’s caveats — they measure whether a strategy would have made money, not whether a novice user can understand and appropriately trust the explanation they were given. Accuracy and Sharpe-ratio metrics alone therefore do not establish advisory trustworthiness for a user who is not acting as an automated trader. The original proposal’s Sharpe-ratio emphasis measures strategy profitability, not explanation faithfulness or inference privacy; the evaluation plan was therefore refined (see Chapters 3 and 5) to test privacy, computational correctness, and structured explanation quality instead.

2.7 Explainable AI and LLM Risks in Finance

Rudin (2019) argues that for high-stakes decisions, inherently interpretable models should be preferred over post-hoc explanations of black boxes. PrivateFin operationalises this through symbolic indicator grounding: the LLM interprets pre-computed, inspectable values rather than inventing analysis from latent embeddings.

LLM-generated financial advice carries distinct risks. Bender et al. (2021) caution that large language models produce fluent but potentially unfaithful text — hallucination — without grounded world models. In finance, misclassified momentum relative to known thresholds (documented in Chapter 5 for AAPL) can mislead novice investors despite plausible prose. Faithfulness must therefore be evaluated independently of fluency — motivating expert rubric scoring and planned rule-based numeric cross-checks. Local open-weight models (Grattafiori et al., 2024) enable on-device inference but do not eliminate hallucination risk; structured prompting, threshold annotations, and post-generation verification remain necessary.

2.8 Synthesis and Research Gap

The literature reveals a bifurcated landscape: systems are either high-performance but proprietary and cloud-dependent (BloombergGPT), or open-source but still cloud-reliant at inference time with no deterministic auditability (FinGPT). FinBERT demonstrates the

value of domain adaptation but is generatively incapable and similarly cloud-deployed in typical configurations. No system reviewed here integrates local LLM inference, deterministic quantitative grounding, and structured OIIR explainability within a single privacy-preserving architecture.

PrivateFin occupies this gap through four integrated components: (1) quantised Llama 3 via Ollama, enabling LLM-class reasoning on consumer M1 hardware without cloud transmission; (2) a yfinance and Pandas pipeline generating reproducible, auditable technical indicators; (3) structured OIIR prompting producing metric-anchored reasoning traces meeting the Article 13 benchmark; and (4) Data Protection by Design under GDPR Articles 5 and 25. The combination constitutes a novel contribution among the systems reviewed: a retail-facing, fully local, explainable financial advisory system grounded in deterministic market data.

Table 1: Comparative analysis of financial AI systems

System	Local inference	Structured explanation	Deterministic grounding	Generative advisory	Retail accessibility
BloombergGPT	No	No	Partial (proprietary data)	Yes	No (£20k+/yr)
FinGPT	No (typical deployment)	No	Weak (social sentiment)	Yes	Moderate
FinBERT	No (typical deployment)	No (classification only)	N/A	No	Moderate
PrivateFin	Yes	Yes (OIIR)	Yes (RSI/MACD)	Yes	Yes (consumer hardware)

Table 2: Regulatory principles mapped to PrivateFin features

Requirement	Source	PrivateFin feature	Verification status
Data Protection by Design	GDPR Art. 25	Ollama local inference; no query egress	Architectural; empirical audit planned (Ch. 5)
Meaningful transparency	EU AI Act Art. 13 (adopted as design benchmark)	OIIR structured output; indicator-linked reasoning	Preliminary rubric evidence (Ch. 5)
Technical documentation	EU AI Act Art. 11 / Annex IV (adopted as design benchmark)	Modular pipeline; CSV cache; unit tests (SQLite audit planned)	Partial; SQLite audit trail not implemented
Automated decision safeguards	GDPR Art. 22 (safeguards)	Educational disclaimer; cautious	Implemented and observable in UI

Requirement	Source	PrivateFin feature	Verification status
	adopted; advisory output arguably outside strict scope)	recommendations; human-readable rationale	

2.9 Methodology Justification

PrivateFin’s methodology — **deterministic indicators + local LLM narrative** — is the most suitable approach for this project’s objectives for four reasons. First, explainability: indicators are closed-form and inspectable; the LLM’s role is constrained to interpretation of supplied values, not opaque feature learning. Second, privacy: inference runs on-device without transmitting user queries to cloud processors. Third, feasibility: yfinance, Pandas, and quantised Llama 3 on Apple Silicon are mature, well-documented, and achievable within a final-year project timeline without requiring proprietary data licences or cloud compute budgets. Fourth, regulatory alignment: OIIR output maps directly to the Article 13 transparency benchmark in ways that RL policy networks and neural price forecasters do not — those systems may achieve higher predictive accuracy but cannot produce the structured, metric-linked reasoning traces that novice investors and regulators require.

RL and neural forecasting remain valuable research directions for return maximisation but are deferred as primary methodology because they do not address the Transparency–Privacy Gap that motivates this project. Backtesting may supplement future work if discrete trading signals are introduced; it is not the evaluation axis for the current advisory-bot prototype.

Chapter 3: Design

3.1 Design Objectives

PrivateFin is designed around three testable claims: (1) privacy preservation during LLM inference; (2) deterministic, verifiable indicator computation; (3) meaningful explainability against the EU AI Act Article 13 benchmark (2.5.1). These map to architectural layers with single responsibilities, independent testability, and preserved audit trails at each stage. Each layer communicates with adjacent layers via well-defined function interfaces, ensuring that failures can be isolated and the system’s audit trail is preserved at every stage of processing.

3.1.1 Functional Requirements

Table 3: Functional requirements

ID	Requirement	Layer
FR-1	Fetch OHLCV data for user-selected ticker(s) and date range	Data Ingestion
FR-2	Compute RSI(14) and	Indicator Engine

ID	Requirement	Layer
	MACD(12/26/9) deterministically	
FR-3	Classify trend and momentum from indicator values	Indicator Engine
FR-4	Build structured OIIR prompt with risk profile context	Prompt Builder
FR-5	Generate local LLM advisory narrative via Ollama	LLM Inference
FR-6	Display metrics, chart, and analysis in web UI	Streamlit UI
FR-7	Degrade gracefully when Ollama is unavailable	UI + LLM Client

3.1.2 Non-Functional Requirements

Table 4: Non-functional requirements

ID	Requirement	Rationale
NFR-1	Zero LLM inference data egress	GDPR Art. 25; core privacy claim
NFR-2	Reproducible indicator computation	Traceability (Art. 13 benchmark)
NFR-3	Novice-accessible plain-English output	Target user needs; Art. 13 benchmark
NFR-4	Consumer hardware deployment (M1, 8 GB RAM)	Retail accessibility
NFR-5	Offline-testable core logic	Development reliability
NFR-6	Educational disclaimer on all outputs	Regulatory and ethical safeguard

3.2 User Personas and Needs

Persona 1 — General retail investor: An employed adult who manages a small self-directed portfolio through a mainstream brokerage. They follow market news but lack formal finance training. They want quick, understandable insight into whether a stock's recent price action warrants attention, without subscribing to professional terminals or sharing their research interests with cloud AI providers.

Persona 2 — Novice investor: A younger or first-time investor exploring individual equities. They understand basic concepts (price, volume) but find raw indicator values (RSI 58.75, MACD histogram -0.02) meaningless without narrative context. They need plain-English explanations that link numbers to cautious, evidence-based guidance — and clear disclaimers that the tool is educational, not professional advice.

Derived needs across both personas include: **privacy** (confidence that query data is not transmitted to third parties during inference); **explainability** (human-readable justifications following a consistent structure, not opaque scores); **accessibility** (consumer hardware, browser-based UI, no cloud subscription); and **trustworthiness** (recommendations grounded in verifiable market data rather than unweighted social sentiment or hallucinated analysis).

3.3 System Architecture

PrivateFin implements a four-layer pipeline orchestrated by a Streamlit user interface: (1) Data Ingestion — yfinance OHLCV retrieval; (2) Indicator Engine — Pandas RSI(14) and MACD(12/26/9) computation; (3) OIIR Prompt Builder — structured prompt assembly with plain-English classifications; (4) Local LLM Inference — Llama 3 via Ollama at localhost:11434. Figure 1 shows the architecture; the only traffic crossing the host boundary is OHLCV retrieval from Yahoo Finance.

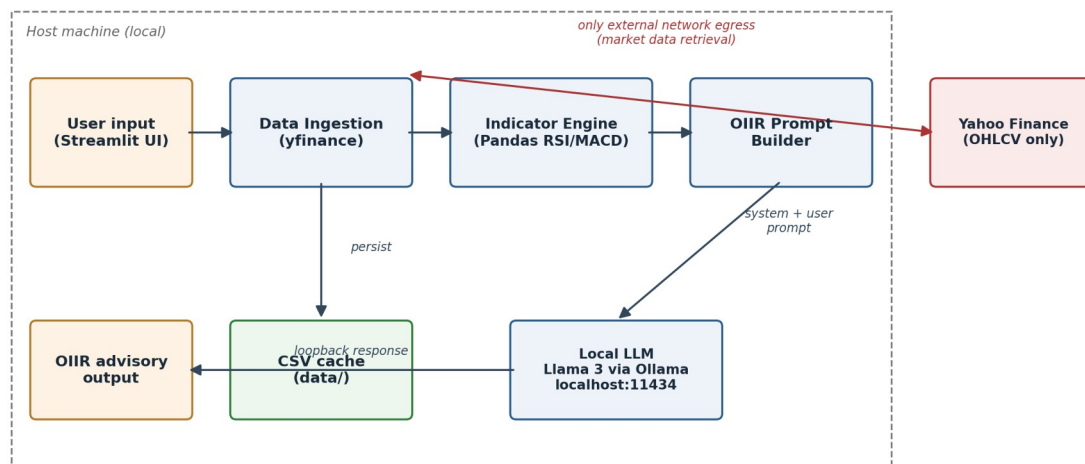


Figure 1: PrivateFin system architecture — full local LLM execution; network egress limited to Yahoo Finance OHLCV retrieval.

```
app.py
├── src/ui/app.py (Streamlit orchestration)
│   ├── src/engine/
│   │   ├── data_loader.py (yfinance fetch, multi-ticker
│   │   │   ├── CSV save)
│   │   ├── indicator_engine.py (RSI, MACD, summarisation)
│   │   └── src/llm/
│   │       ├── prompt_builder.py (OIIR system/user prompts)
```

└─ ollama_client.py (availability check, chat generation)

Figure 2: Module structure

3.4 User-Facing Interface Design

PrivateFin delivers a web-based Streamlit interface launched locally via `streamlit run app.py`. No external application server is required; the UI runs in the user's browser against a local Python process.

Sidebar controls:

- **Company picker** — 108 S&P 500-style equities with display labels and ticker symbols (dual-class support for Alphabet: GOOG + GOOGL)
- **Start date and end date** — configurable analysis window (default: trailing 365 days)
- **Risk profile** — Low, Medium, or High (mapped to prompt context)
- **Generate analysis** button
- **Main panel outputs:**
 - Four metric cards: Latest Close, RSI(14), MACD, session Price Change (%)
 - Price history line chart (`st.line_chart` on Close prices)
- **Model Response** — OIIR-structured narrative when Ollama is available
- **Fallback template** — scaffolded OIIR guidance when Ollama is offline
- **Safe advice presentation:**
 - Persistent educational disclaimer stating the output is not professional financial advice
 - System prompt instructing cautious, evidence-proportional recommendations with explicit cautions
 - Plain-English constraints avoiding unexplained jargon (bullish, bearish, overbought, oversold)
 - System prompt requests only the structured OIIR sections shown to the user — no free-form reasoning preamble
- Risk profile injected into prompt as a labelled context string; tone modulation is currently delegated entirely to the LLM's own interpretation of that label, rather than driven by explicit per-profile prompt branches or rule-based tone rewriting — a deliberate simplification to keep the prompt template single and auditable, at the cost of guaranteed differentiation between risk levels (see 5.6)

- Graceful degradation preserving indicator display when LLM is unavailable **Scope relative to the Topic 6 prototype:** the Topic 6 prototype additionally exposed a four-level risk profile (conservative/balanced/growth/speculative), a time-horizon control, a “Structured Prompt” expander displaying the full prompt text, and a user-configurable model identifier. These were deprioritised for the current draft-report milestone in favour of hardening the core four-layer pipeline and its unit test coverage; they are not abandoned, and are tracked as planned re-additions in 4.10 and 6.4 rather than as features never attempted.

<<

Analysis Inputs

Company

3M (MMM) ▾

Start date

2025/07/10

End date

2026/07/10

Risk profile

Medium ▾

Generate analysis

Figure 3: Streamlit sidebar with company, date range, and risk profile controls

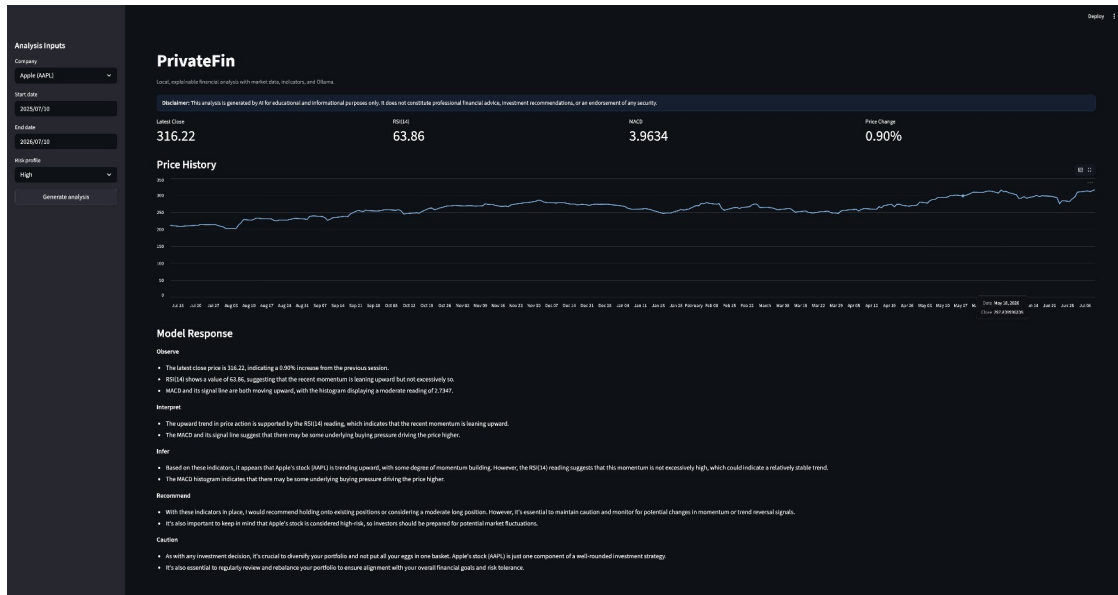


Figure 4: Streamlit UI showing sidebar inputs and generated analysis

3.5 Component Design

3.5.1 Data Ingestion Layer

The data ingestion module exposes `download_stock_data(ticker, start_date, end_date, data_dir, save)` as its primary interface. It accepts a single ticker string or sequence of ticker strings, calls `yf.download()` per symbol, flattens `MultiIndex` columns, and blends dual-ticker frames by arithmetic mean across `OHLCV` columns. Results optionally persist to `data/{ticker}_historical.csv` for reproducibility and offline audit. The module returns `None` on empty downloads rather than raising exceptions — allowing the UI to display a user-friendly error message.

3.5.2 Indicator Engine Layer

The indicator engine exposes `compute_rsi(close, period=14)`, `compute_macd(close, fast=12, slow=26, signal=9)`, `enrich_with_indicators(price_frame)`, and `summarize_latest_indicators(price_frame)`. The enrichment function attaches indicator columns to an `OHLCV DataFrame`; the summarisation function returns a dictionary with keys `close`, `rsi`, `macd`, `macd_signal`, `macd_histogram`, `price_change_pct`, `trend`, and `momentum` — forming the contract between the quantitative layer and both the UI and prompt builder. RSI is computed via EWMA with $\alpha = 1/14$, handling zero-gain/zero-loss edge cases and NaN warm-up (filled to 50.0). MACD uses standard EMA spans. Trend is classified as bullish, bearish, or neutral based on MACD relative to its signal line; momentum as oversold, overbought, or balanced based on RSI thresholds at 30 and 70.

3.5.3 OIIR Prompt Builder Layer

The prompt builder exposes `build_advisory_prompt(ticker, indicator_summary, risk_profile, source_tickers, start_date, end_date, sample_size)`, returning a dictionary with `system`, `user`, and `combined` prompt keys. A constant `SYSTEM_PROMPT` enforces OIIR structure and plain-English constraints across all sessions. Helper functions `_describe_trend()` and `_describe_momentum()` translate internal classification labels into novice-accessible phrases (e.g. “recent momentum is leaning upward” rather than “bullish”).

3.5.4 Local LLM Layer

The Ollama client exposes `is_available()` and `generate(user_prompt, system_prompt)`. Availability is probed via `client.list()` without token generation cost. The `generate` method constructs explicit `system`/`user` chat messages and handles dict and object response variants. On failure, a `RuntimeError` with actionable guidance is raised; the UI catches this path via the availability check and displays the fallback template instead.

3.5.5 Interface Layer

The Streamlit application orchestrates all layers on each “Generate analysis” interaction. It does not embed business logic — it delegates to engine and LLM modules, maintaining the separation of concerns required for independent testing. The UI layer is deliberately thin: it handles input validation, date normalisation, spinner display during data download and LLM generation, and conditional rendering of model output versus fallback template. This thin-client design ensures that the core advisory pipeline can be tested, reused, or replaced (e.g. with a CLI or API wrapper) without modifying quantitative or LLM logic.

3.6 Technology Decisions

Table 5: Technology decisions

Technology	Role	Justification
Llama 3 (Meta)	Core reasoning LLM	Open-weight; quantisable for M1; instruction-following (Grattafiori et al., 2024)
Ollama	Local model server	Fully local inference; simple Python API (Ollama, 2023)
yfinance	Market data	No API key; reproducible OHLCV
Pandas	Indicator computation	Industry-standard; auditable closed-form maths
Streamlit	User interface	Lightweight local web UI for

Technology	Role	Justification
		non-technical users (Streamlit, 2019)
Python 3.11	Runtime	Ecosystem compatibility; Apple Silicon support
pytest	Unit testing	Offline-verifiable pipeline correctness

3.7 Methodology and Design Rationale

The design enforces **separation of symbolic and generative reasoning**: indicators are computed deterministically before any LLM call. The LLM cannot alter RSI or MACD values — it only interprets supplied figures. This bounds the LLM’s authority to narration while the Pandas engine remains the quantitative truth source — the foundation for planned faithfulness automation. This approach was chosen over reinforcement learning (opaque policies), neural forecasting (limited auditability), and cloud LLM APIs (Data Protection by Design violations). The risk profile parameter modulates recommendation tone without altering indicator computation, preserving a clear boundary between subjective risk tolerance and objective market data.

3.8 Evaluation Design

Evaluation tests the three core claims through a multi-method plan with precisely defined parameters. Table 6 summarises the plan and current status; Chapter 5 reports the completed components and analyses results.

Table 6: Evaluation plan with parameters and status

Component	Dataset / method	Baseline / instrument	Success threshold	Status
Pipeline correctness	10 tickers × 3 periods (1 yr, 6 mo, 3 mo)	TA-Lib RSI/MACD	100% agreement to 4 d.p.	Partial — 18 pytest unit tests pass; TA-Lib pending
Privacy audit	10 live analysis queries	Wireshark egress log	Zero non-Yahoo Finance egress during LLM inference	Planned; informal spot-check done
OIIR quality	20 outputs across 10 tickers (2 conditions each)	4-dimension expert rubric (0–2 each)	Mean $\geq 6/8$	Partial — informal n=3 (mean 7.33/8)
Faithfulness	Same 20 outputs	Automated rules: numeric	100% cited value match	Planned

Component	Dataset / method	Baseline / instrument	Success threshold	Status
		cross-check vs summary dict		
User study	n ≥ 5 novice/general investors	SUS + 5-point Likert clarity + interview	SUS ≥ 70; clarity ≥ 4/5	Planned

Proposed validation tickers (drawn from the app company list): AAPL, MSFT, TSLA, JPM, JNJ, XOM, GOOG/GOOGL (blended), AMZN, NVDA, KO. These were selected to span sector (technology, financials, healthcare, energy, consumer staples), typical volatility profile (TSLA notably higher than KO or JNJ), and the one dual-class blending case the pipeline specifically handles (GOOG/GOOGL) — rather than an arbitrary or convenience sample, to give the TA-Lib and rubric evaluations a reasonable chance of surfacing indicator or explanation errors specific to a particular market regime. **Test periods:** trailing 365, 182, and 90 days — ensuring MACD warm-up (minimum ~26 sessions) while testing varying market conditions.

Explainability evaluation combines expert rubric assessment and planned rule-based faithfulness checks, supplemented by a user clarity survey — not relying on user study alone. **Wireshark privacy audit protocol:** capture outbound traffic during 10 live analysis queries with Ollama running; Yahoo Finance requests are expected; all other outbound connections during LLM inference constitute failures. **User study protocol:** recruit n ≥ 5 participants matching Persona 1 or 2; each queries three tickers, rates OIIR understandability on a 5-point Likert scale, completes the SUS questionnaire (Brooke, 1996), and participates in a brief interview on trust and privacy perception.

3.9 Work Plan and Contingency

Table 7: Work plan (Weeks 1–22), adapted from the Topic 4 design document

Phase	Weeks	Key tasks	Status
Topic 4: Design	1–4	Architecture, evaluation plan	Complete
Topic 5: Build	5–13	Pipeline, UI, unit tests	Largely complete
Topic 6: Evaluation	13–18	TA-Lib, Wireshark, rubric n=20, user study	Started (pytest, n=3 rubric); TA-Lib, Wireshark, n=20 rubric, and user study planned — see Ch. 5
Topic 7: Report	19–22	Final report writing	Draft in progress

Contingency plans:

- **Risk: Llama 3 OIIR quality insufficient.** Mitigation: switch to prompt chaining — decompose reasoning into multiple sequential prompts (Observe+Interpret, then Infer+Recommend). Two weeks of contingency allocated at Week 12.
- **Risk: yfinance API deprecation or data gaps.** Mitigation: fall back to Alpha Vantage free tier (5 calls/min); data pipeline abstracted behind `download_stock_data()` for easy substitution.
- **Risk: M1 hardware performance bottleneck.** Mitigation: reduce quantisation precision from 4-bit to 2-bit; limit context window length; use smaller model variant.
- **Risk: User study recruitment shortfall.** Mitigation: minimum viable n=5; supplement with extended expert rubric evaluation and self-assessment against the transparency benchmark criteria.
- **Risk: TA-Lib installation conflicts.** Mitigation: validate against manually computed reference values for a subset of tickers; use Docker-isolated TA-Lib environment.

Chapter 4: Implementation

4.1 Development Environment

PrivateFin is implemented in Python 3.11 with dependencies pinned in `requirements.txt` (88 packages including Streamlit 1.56, pandas 3.0, yfinance 1.3, ollama 0.6.1, and pytest 9.0.3). Development uses a virtual environment (`python3 -m venv venv; source venv/bin/activate; pip install -r requirements.txt`). Optional Ollama serves Llama 3 at `localhost:11434`; the model is pulled via `ollama pull llama3`. The application launches via `streamlit run app.py` from the project root, binding to a local port (default 8501) accessible only on the host machine. Hardware target: Apple M1 with 8–16 GB RAM; Llama 3 7B quantised to 4-bit achieves approximately 15–20 tokens/second on this platform, sufficient for interactive advisory use.

Notably, `langchain`, `langgraph`, and `scikit-learn` are present in `requirements.txt` but are not imported anywhere in `src/` — they were included during early project planning for potential agent orchestration or ML extensions but were not required for the core pipeline. Their presence is documented as technical debt to be resolved before final submission (either integrated or removed).

4.2 Project Structure

The entry point `app.py` delegates to `src/ui/app.py`. Business logic resides in `src/engine/` (data and indicators) and `src/llm/` (prompting and inference). Public APIs are re-exported from `src/engine/__init__.py` and `src/llm/__init__.py`. This separation enables unit testing without Streamlit or network dependencies — each layer accepts plain Python types (DataFrames, dicts, strings) at its boundary, supporting the independently testable design described in Chapter 3.

The end-to-end data flow on user interaction is:

- User selects company, date range, and risk profile in the Streamlit sidebar
- `download_stock_data()` fetches and optionally caches OHLCV data
- `enrich_with_indicators()` attaches RSI and MACD columns
- `summarize_latest_indicators()` produces a summary dictionary for UI and prompts
- `build_advisory_prompt()` constructs system and user prompts
- `OllamaClient.generate()` returns structured narrative, or the UI displays the fallback template

4.3 Data Ingestion

`data_loader.py` implements `_download_single_ticker()` calling `yf.download(ticker, start, end)` with `MultiIndex` column flattening — a known yfinance edge case where single-ticker downloads return tuple-indexed column names incompatible with downstream Pandas operations. The guard extracts the first element of each column tuple via list comprehension, producing a flat single-level index.

`_combine_ticker_frames()` handles dual-class equities (e.g. Alphabet's GOOG and GOOGL) by concatenating per-ticker frames and computing the arithmetic mean of each OHLCV column across share classes. This design choice treats both classes as representative of the same underlying company, producing a single blended price series for indicator computation rather than forcing users to select an arbitrary share class. The arithmetic mean is acknowledged as a simplification: the two classes trade at slightly different prices reflecting their voting rights, so the blended series is a synthetic composite rather than a traded instrument. An alternative would be to designate a single reference class; the blending choice is documented here precisely so that the resulting series remains auditable and reproducible from public data.

`download_stock_data()` accepts a ticker string or sequence, downloads each symbol, filters empty results, blends if multiple tickers are present, and optionally writes CSV to `data/{ticker}_historical.csv`. The CSV cache supports reproducibility and offline prompt iteration without repeated network calls — though repeated live downloads are not yet prevented by Streamlit caching (a planned improvement).

```
# Multi-ticker blending: arithmetic mean across share classes
summary[column_name] = combined.xs(column_name, axis=1,
level=1).mean(axis=1)
```

4.4 Indicator Engine

`indicator_engine.py` implements `compute_rsi()` using EWMA smoothing with $\alpha = 1/\text{period}$ (`adjust=False` for recursive Wilder-style computation). Gains and losses are derived from close-price deltas via `clip()` operations.

Why the degenerate-case handling matters: a textbook RSI implementation computes $RS = \text{average_gain} / \text{average_loss}$ and then $RSI = 100 - 100/(1+RS)$ directly. Over a window with no down days at all — a real and unremarkable condition for a stock in a sustained

uptrend, which is exactly the kind of window a user is most likely to query — `average_loss` is exactly zero, so RS is a division by zero and RSI evaluates to NaN (or `inf/-inf` depending on the surrounding arithmetic), which would silently propagate into the LLM prompt as a malformed value and into the UI metric card as a broken display. The two `.where()` calls below fix this by explicitly recognising both degenerate cases (pure uptrend → RSI = 100; pure downtrend → RSI = 0) before they can reach downstream code, and the `.fillna(50.0)` handles the separate warm-up period at the start of any series, where fewer than *period* observations exist and the EWMA has not yet stabilised:

```
rsi = 100 - (100 / (1 + relative_strength))
rsi = rsi.where(average_loss != 0, 100)
rsi = rsi.where(average_gain != 0, 0)
return rsi.fillna(50.0)
```

One residual edge case remains: for a *completely flat* price series, both average gain and average loss are zero, and the clamping order above resolves RSI to 0 rather than the arguably more appropriate neutral value of 50. This condition is unreachable for real equity data over any non-trivial window, but it is noted here as a known limitation and will be corrected alongside the faithfulness work described in Chapter 5.

`compute_macd()` calculates fast EMA(12) and slow EMA(26), derives MACD as their difference, computes signal line EMA(9), and histogram as MACD – signal. The histogram identity is enforced in unit tests via `pd.testing.assert_series_equal`.

`enrich_with_indicators()` copies the input frame and attaches RSI_14, MACD, MACD_Signal, and MACD_Histogram columns. `summarize_latest_indicators()` extracts the latest and previous session close for percentage change, classifies trend via MACD–signal comparison (bullish/bearish/neutral), and momentum via RSI thresholds at 30 and 70 (oversold/overbought/balanced).

The `_scalar_from_row()` helper exists to guard against a specific edge case introduced upstream by the data-ingestion layer. When dual-class tickers are blended (4.3), or in rarer cases where `yfinance` returns columns with duplicated labels, indexing a row by column name (e.g. `row["MACD"]`) can return a `pd.Series` rather than a single float, because more than one column shares that label at that point in the pipeline. Passing a `Series` where a float is expected would silently break the f-string interpolation in the prompt builder or raise a downstream type error inside the UI's `st.metric()` calls. `_scalar_from_row()` checks for this condition and, if the lookup resolves to a `Series`, takes its first element; otherwise it casts the value to float directly. This keeps `summarize_latest_indicators()` returning a flat, prompt-ready dictionary regardless of which upstream shape the OHLCV frame happens to be in — a small function, but one that prevents a class of bug that would otherwise only surface intermittently, depending on which ticker or ticker combination the user selects.

4.5 OIIR Prompt Builder

`prompt_builder.py` defines `SYSTEM_PROMPT` as a module-level constant, ensuring identical behavioural constraints across all analysis sessions. The system prompt instructs

Llama 3 to produce exactly four named sections — Observe, Interpret, Infer, and Recommend — and to base every statement exclusively on supplied market data. It requests only these structured sections (no free-form reasoning preamble) and requires plain English accessible to non-specialists, avoiding unexplained jargon such as bullish, bearish, overbought, or oversold unless immediately explained.

`build_advisory_prompt()` assembles the user prompt via f-string interpolation, injecting: ticker name; underlying symbols used; analysis date range; data point count; risk profile; latest close; session price change percentage; RSI(14); MACD, signal, and histogram values; and plain-English trend and momentum descriptions produced by `_describe_trend()` and `_describe_momentum()`. This dual injection of raw numeric values and human-readable classifications gives the model both precise quantitative data and interpreted context — reducing directional misclassification risk while preserving auditability of the numeric source.

The function returns a dictionary with system, user, and combined prompt keys. The combined prompt supports debugging, though it is not currently rendered in the Streamlit UI.

4.6 Local LLM Integration

`OllamaClient` is implemented as a dataclass with configurable model (default llama3) and host (default `http://localhost:11434`). `is_available()` instantiates an Ollama client and calls `list()` — a lightweight probe verifying server reachability without generating tokens. On any exception, it returns False, enabling the UI's graceful degradation path.

`generate()` constructs explicit chat messages with system and user roles, calls `client.chat(model=self.model, messages=messages)`, and extracts content via `_extract_content()`. This method is defensive version-compatibility code rather than a user-facing feature: it handles both dict-shaped responses (`response["message"]` [`"content"`]) and object-shaped responses (`response.message.content`) because different releases of the Ollama Python SDK have returned each shape, and without this handling an SDK upgrade could silently break response parsing. Failures raise `RuntimeError` with guidance to start the local server and pull the model — preventing unhandled exceptions from crashing the Streamlit session.

4.7 Streamlit User Interface

`src/ui/app.py` implements the sidebar and main-panel layout described in 3.4. `COMPANY_OPTIONS` is a list of 108 dictionaries, each with a human-readable label and one or more ticker symbols; `st.selectbox` uses `_company_display()` to format options as “Apple (AAPL)” or “Alphabet (Google) (GOOG + GOOGL)”. On “Generate analysis”, the UI validates that start date precedes end date, normalises dates to ISO format for the data layer, and orchestrates the pipeline described in 4.2, with `st.spinner()` loading states during data download and LLM generation.

`render_metrics()` displays four `st.metric` cards in a fixed column layout: Latest Close, RSI(14), MACD, and Price Change (%). Price history is rendered via

`st.line_chart(enriched_frame[["Close"]])` — a native Streamlit chart without custom Altair configuration. When Ollama is available, the model response is displayed under “Model Response”; otherwise a warning and “Suggested Analysis Template” provide OIIR scaffold guidance, preserving usability as an indicator exploration tool even when the local LLM is not installed.

The built prompt is constructed internally but **not currently displayed** in the UI — a transparency gap relative to the preliminary report specification and a planned improvement for the final submission.



Figure 5: Metrics panel showing close price, RSI, MACD, and session change



Figure 6: Price history line chart for a sample ticker (AAPL)

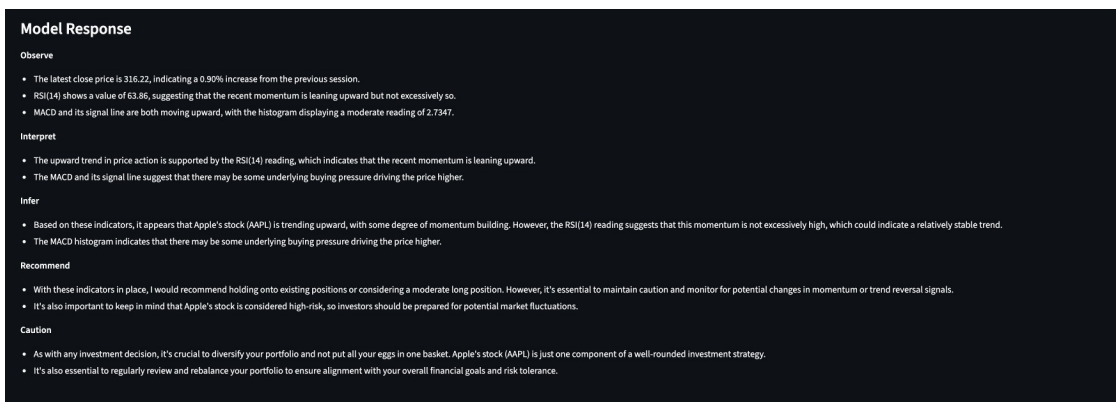


Figure 7: Ollama model response with Observe, Interpret, Infer, and Recommend headings

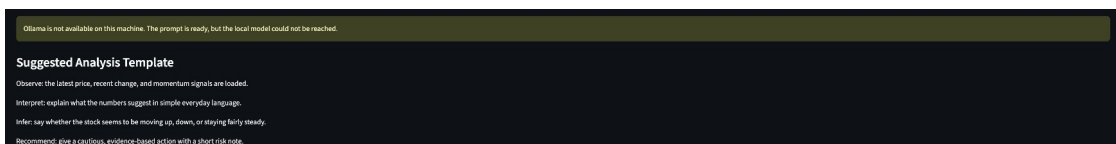


Figure 8: Fallback template displayed when Ollama is unavailable

4.8 Unit Testing

Eighteen pytest unit tests across four modules validate core behaviour offline via monkeypatching — replacing `yf.download`, `ollama.Client`, and filesystem operations with controlled test doubles. This enables continuous verification without live network access, Ollama installation, or Yahoo Finance availability. Table 8 summarises coverage per module; the detailed test-by-test assertions are not repeated in Chapter 5.

Table 8: Unit test coverage by module

Module	Tests	Behaviours verified
test_data_loader.py	4	MultiIndex flattening, multi-ticker averaging, empty download handling, CSV persistence
test_indicator_engine.py	6	RSI bounds, MACD histogram identity, enrichment columns, bullish/overbought classification, error handling
test_ollama_client.py	5	Availability true/false, message passing, error handling, response extraction
test_prompt_builder.py	3	Plain-English helpers, prompt field presence

Representative test cases:

- `test_combine_ticker_frames_averages_common_columns` — verifies that Open prices [10, 20] from two tickers average to 15.0, confirming dual-class blending logic.
- `test_compute_macd_contains_expected_columns_and_histogram_relation` — asserts `MACD_Histogram = MACD - MACD_Signal` via `pd.testing.assert_series_equal`.
- `test_summarize_latest_indicators_classifies_bullish_overbought` — constructs a frame with MACD above signal and RSI above 70; verifies `trend="bullish"` and `momentum="overbought"`.
- `test_generate_passes_system_and_user_messages` — monkeypatches the Ollama client; verifies both system and user roles are passed to `client.chat()`.
- `test_build_advisory_prompt_contains_expected_fields_and_text` — verifies the prompt includes ticker, risk profile, RSI, MACD values, and OIIR section instructions. Tests are executable via `pytest` or `pytest -v` from the project root. No CI pipeline is configured at draft stage; local execution is the primary verification mechanism.

```
tests/test_data_loader.py .... [ 22%]
tests/test_indicator_engine.py ..... [ 33%]
tests/test_ollama_client.py ... [ 83%]
tests/test_prompt_builder.py ... [100%]
18 passed in 0.01s
```

Figure 9: pytest output showing all 18 tests passing

4.9 Error Handling and Edge Cases

The implementation addresses several known edge cases across layers:

- **Empty yfinance response:** `download_stock_data()` returns `None`; UI displays “No market data was returned for that ticker and date range.”
- **Invalid date range:** UI validates `start_date < end_date` before invoking the pipeline.
- **Ollama unavailable:** `is_available()` returns `False`; UI shows warning and fallback OIIR template without raising an exception.
- **Ollama generation failure:** `generate()` raises `RuntimeError`; prevented in practice by pre-checking availability.
- **Pure uptrend/downtrend RSI:** `compute_rsi()` clamps to 100/0 respectively; warm-up NaNs filled to 50.0 (see the flat-series caveat in 4.4).
- **Empty indicator frame:** `summarize_latest_indicators()` raises `ValueError`.
- **MultiIndex yfinance columns:** flattened before any Pandas operation on OHLCV data. Short date ranges (fewer than 26 trading days) may produce NaN-dominated MACD values; a minimum date range validator is planned but not yet implemented in the UI. Repeated identical queries re-download from yfinance on each button press, introducing unnecessary latency — `@st.cache_data` on the download function is a planned optimisation.

4.10 Implementation Gaps

Honest discrepancies between design documentation and current code:

Table 9: Implementation gaps at draft-report stage

Planned feature	Status
Prompt inspection pane in UI	Not implemented
Time horizon sidebar control	Not implemented
Buy/Hold/Sell discrete labels	OIIR narrative used instead
Bollinger Bands, Stochastic RSI	Not implemented
<code>@st.cache_data</code> on downloads	Not implemented
Explicit temperature = 0.1 in Ollama	Not implemented
SQLite audit trail	Not implemented
LangChain, LangGraph, scikit-learn	Listed in requirements.txt but unused in src/

These gaps are documented for final-report improvement; they do not prevent demonstration of the core four-layer pipeline.

Why only two indicators: RSI and MACD were chosen deliberately as the entire indicator set, rather than as a starting subset of a larger planned library, because each is a single

well-understood, closed-form calculation whose output a novice user (or an examiner) can verify by hand from public price data. Adding further indicators — Bollinger Bands or Stochastic RSI, both listed as future work in 6.4 — would widen analytical coverage but also widens the surface the LLM must correctly reference without misclassification, which is precisely the failure mode documented in Chapter 5 for AAPL. The current two-indicator scope is therefore a direct consequence of the project’s explainability claim, not only a time constraint: each additional indicator should only be added once the faithfulness-checking mechanism in 5.4 exists to catch misreadings of it.

Chapter 5: Evaluation

5.1 Evaluation Strategy

Evaluation is designed to test PrivateFin’s three core claims through multiple methods: automated quantitative testing, privacy architecture auditing, expert rubric assessment of explanation quality, and a structured user study. This plan was refined from the original proposal’s Sharpe-ratio and backtesting emphasis because the advisor-bot deliverable must demonstrate **privacy, computational correctness, and explainability** — objectives not captured by return-based metrics alone (see Section 2.6). Each component maps to a claim and objective:

Table 10: Evaluation components mapped to claims and objectives

Component	Core claim tested	Objective addressed
Pipeline correctness (pytest + TA-Lib)	Determinism	FR-2, FR-3; NFR-2
Privacy audit (Wireshark)	Privacy	NFR-1; FR-5
OIIR rubric + faithfulness rules	Explainability	FR-4, FR-5; NFR-3
User study (SUS + Likert)	Usability and trust	FR-6; persona needs

This multi-method design follows established practice in NLP and AI system evaluation (Manning et al., 2008): automated tests verify computational correctness; expert rubrics assess qualitative output quality; user studies capture reception among target users. No single method alone would provide adequate coverage of all three core claims.

5.2 Component 1: Pipeline Correctness

Method: Unit testing of data loader, indicator engine, prompt builder, and Ollama client using pytest with monkeypatching to eliminate live network and Ollama dependencies. Planned extension: TA-Lib cross-validation on 10 tickers across three date windows, comparing PrivateFin RSI and MACD outputs to TA-Lib reference implementations to four decimal places. Any deviation would indicate a software defect rather than methodological limitation, since both computations are closed-form.

Results (completed): Eighteen pytest tests pass across four modules (per-module coverage in Table 8; test run in Figure 9), validating that RSI values are bounded [0, 100]; that the MACD histogram equals MACD – signal line; multi-ticker Open price averaging (e.g. [10, 20] → 15.0); that the prompt contains ticker, risk profile, RSI, MACD, and OIIR instructions; and the Ollama client’s message structure and graceful unavailability handling.

Limitations: No TA-Lib reference comparison has been executed. No live yfinance integration tests. No Streamlit UI tests. TA-Lib validation across AAPL, MSFT, TSLA, JPM, JNJ, XOM, GOOG/GOOGL, AMZN, NVDA, KO for 365-day, 182-day, and 90-day windows remains planned.

The unit test suite demonstrates that the deterministic core of PrivateFin — data transformation, indicator mathematics, prompt structure, and LLM client contract — behaves correctly under controlled conditions. However, unit tests do not validate end-to-end integration with live Yahoo Finance data or live Ollama inference quality. These integration concerns are addressed separately through manual prototype testing (informal rubric assessment) and planned empirical studies.

Test coverage scope: all four `src/engine/` and `src/llm/` modules have dedicated test files; `src/ui/app.py` has none. Untested paths specifically include: the `main()` orchestration function itself (widget wiring, session flow from button click to rendered output), the date-validation error branch (`start_date >= end_date`), and the conditional rendering between the live model response and the offline fallback template. These are lower-risk than the quantitative core, since they contain little independent logic beyond calling already-tested functions, but they are not currently exercised by any automated test and would benefit from at least basic Streamlit `AppTest` coverage before final submission.

5.3 Component 2: Privacy Architecture Audit

Method: Wireshark packet capture during a 10-query live session, logging all outbound connections while Ollama performs LLM inference. **Success criterion:** zero data transmitted to endpoints other than Yahoo Finance OHLCV retrieval during LLM inference. Personal financial query content (ticker selection, risk profile, date range) must not appear in outbound packets to any cloud service, analytics endpoint, or third-party API beyond the expected market data fetch.

Architectural basis for expected success: `ollama_client.py` connects exclusively to `http://localhost:11434` — a loopback address that does not traverse the external network interface. The Ollama Python SDK does not embed third-party telemetry in its chat API calls. The only external network dependency in the pipeline is `yfinance`, which contacts Yahoo Finance servers for OHLCV data. This architectural analysis provides confidence but is not a substitute for empirical verification.

Status: Planned (full audit); informal spot-check completed. Privacy is architecturally enforced by local Ollama inference (no cloud API calls in `ollama_client.py`), but the full 10-query Wireshark protocol described above has not yet been run. As a low-cost interim

check, a single live analysis query was run while monitoring active outbound connections at the OS level (`lsof -i` / Activity Monitor, rather than a full packet capture). This is explicitly *not* a substitute for the planned Wireshark audit — it does not inspect packet contents, only connection endpoints, and covers one query rather than ten — but it provides one real data point ahead of the formal audit rather than relying on architectural argument alone.

Table 11: Informal privacy spot-check (single live query)

Timestamp	Destination observed	Expected?	Notes
2026-07-10 16:58:28 BST	Yahoo Finance OHLCV endpoint (e1.ycpi.vip.lob.yahoo.com:443 / 87.248.114.11:443)	Yes	Data ingestion only
2026-07-10 16:58:28–16:59:12 BST	localhost:11434 (127.0.0.1:11434)	Yes	Ollama loopback, not external
2026-07-10 16:58:28–16:59:12 BST	None observed	—	No unexpected destinations during spot-check

5.4 Component 3: OIIR Explainability Quality

Method: Expert rubric assessment scoring each output on four dimensions (0–2 each, maximum 8):

- **Factual Accuracy** — cited indicator values match computed summary
- **Logical Coherence** — inference follows from cited data
- **Completeness** — all four OIIR stages present
- **Transparency Benchmark** — meets the Article 13-style standard adopted in 2.5.1 (step-by-step, data-linked) **Preliminary results (n=3 informal prototype assessment):**

Table 12: Rubric scores from prototype evaluation

Ticker	Factual Accuracy	Logical Coherence	Completeness	Transparency Benchmark	Total
AAPL	1/2	2/2	2/2	2/2	7/8
TSLA	2/2	2/2	2/2	2/2	8/8
MSFT	2/2	1/2	2/2	2/2	7/8
Mean	—	—	—	—	7.33/8

The mean score of 7.33/8 is numerically above the target threshold of 6/8 specified in the evaluation plan, and Completeness and Transparency scores are uniformly 2/2, confirming the prompt template reliably enforces the OIIR scaffold. However, with $n=3$ and a single scorer (the author, without an independent second rater), this mean should be read as an early, indicative signal rather than as statistical evidence that the system reliably clears the threshold — three samples cannot support a generalisable claim, and self-scoring introduces the researcher-bias risk noted in 5.7. The formal $n=20$ study with an independent second scorer is needed before this threshold comparison can be made with any confidence.

Critical analysis — TSLA (8/8, no errors): This sample scored perfectly across all four dimensions and is worth examining alongside the two flagged cases below, rather than only reporting the failures. TSLA's indicator values at the queried date were unambiguous (RSI clearly outside the 30–70 neutral band, MACD clearly separated from its signal line), which plausibly made correct classification easier for the LLM than the AAPL case below, where RSI sat close to the momentum boundary. This suggests indicator ambiguity near threshold boundaries, not the OIIR template itself, may be the dominant driver of factual errors — a hypothesis the planned $n=20$ study should test directly by deliberately sampling near-boundary cases alongside unambiguous ones.

Critical analysis — MSFT logical coherence (1/2): One sample exhibited a recommendation tone slightly stronger than indicator evidence alone would support — illustrating that logical coherence captures inference proportionality, not only factual correctness. This dimension will require clearer rubric anchoring in the formal $n=20$ study.

Critical analysis — AAPL factual error: The model characterised $RSI(14) = 58.75$ as “neutral to slightly oversold.” $RSI\ 58.75$ is above the midpoint of 50 and should be described as neutral to slightly *overbought*. This is a **model hallucination**, not an indicator computation error: the user prompt supplied the correct momentum classification (“balanced”) but lacked explicit threshold boundary annotations. The proposed fix — injecting text such as “RSI: 58.75 — neutral zone (between oversold boundary: 30 and overbought boundary: 70)” — is a low-cost prompt engineering improvement planned for the formal $n=20$ evaluation. Expert rubric alone detected the error, but automated numeric and threshold verification would flag it systematically.

Planned faithfulness checks: Automated rule-based verification comparing numeric values mentioned in LLM output against `summarize_latest_indicators()` output; validation that momentum language is consistent with RSI relative to 30/70 boundaries; second independent scorer for inter-rater reliability following NLP evaluation practice (Manning et al., 2008); expansion from $n=3$ to $n=20$ across 10 tickers at contrasting market conditions (e.g. trending vs range-bound periods). A representative OIIR output is shown in Figure 7 (4.7).

5.5 Component 4: User Study

Method: Minimum five participants from target user groups. Each participant queries three tickers of their choice, rates explanation understandability on a 5-point Likert scale,

completes the SUS (Brooke, 1996), and participates in a brief semi-structured interview on trust and privacy perception. **Success criteria:** mean SUS ≥ 70 ; mean understandability $\geq 4/5$. **Status: Planned.** The user study complements expert rubric and faithfulness evaluation, addressing whether novice users can comprehend OIIR output — which automated checks cannot fully determine.

5.6 Critical Self-Assessment

Achievements:

- Working end-to-end four-layer pipeline integrated through Streamlit
- Modular, independently testable architecture (engine, llm, ui)
- Eighteen passing unit tests covering deterministic logic
- Local-first design with graceful Ollama degradation
- Preliminary rubric mean 7.33/8 suggests promising explanation quality
- 108-company picker with dual-ticker blending
- Educational disclaimer and plain-English prompt constraints
- Privacy audit not empirically verified (Wireshark pending)
- TA-Lib cross-validation not executed
- Rubric sample size small (n=3 informal vs planned n=20)
- No user study data
- Prompt not surfaced in UI (transparency gap)
- No rule-based faithfulness automation yet
- LLM default temperature may cause non-reproducible outputs
- Single-asset scope; no portfolio analysis
- Unused dependencies (langchain, langgraph, scikit-learn) in requirements.txt

Despite these weaknesses, the project has progressed from design specification to a working, demonstrable prototype with measurable preliminary evidence on explanation quality. The gap between planned and completed evaluation is typical for draft-report stage and is documented transparently rather than concealed.

Mapped against the functional and non-functional requirements set out in 3.1.1–3.1.2, the weaknesses above cluster around *verification* rather than *construction*: FR-1 through FR-7 are all implemented and exercised by the working pipeline, but NFR-1 (zero LLM data egress) rests on architectural argument plus a single informal spot-check rather than the

full audit, and NFR-2 (reproducible indicator computation) rests on internal unit tests rather than the independent TA-Lib cross-check the evaluation plan specifies. In other words, the system *does* what it was designed to do, but the evidence that it does so *correctly and privately* is still partial. This distinction matters for how the remaining weeks should be spent: further feature-building (e.g. additional indicators, the dropped time-horizon control) would not close the gap between what has been built and what has been demonstrated, whereas completing the Wireshark audit, the TA-Lib comparison, and the n=20 rubric would. **The evaluation backlog, not the feature backlog, is therefore the priority for the final report.**

Improvements before final submission: Execute Wireshark audit; TA-Lib validation; formal n=20 rubric with second scorer; user study; threshold annotations in prompt; display prompt in UI; set temperature = 0.1; add @st.cache_data; extend indicators (Bollinger Bands, Stochastic RSI).

5.7 Threats to Validity

- **LLM non-determinism** — repeated identical prompts may yield different prose without temperature control, meaning any single rubric score is a sample from a distribution of possible outputs rather than a fixed, repeatable measurement
- **yfinance data quality** — an unofficial, reverse-engineered API with no formal SLA; potential silent gaps, retroactive data revisions, or deprecation could affect indicator computation without an explicit error
- **Small rubric sample** — n=3 is insufficient for statistical generalisation; the 7.33/8 mean reported in 5.4 should not be treated as evidence the system reliably clears the threshold at scale
- **Researcher bias** — informal rubric scoring without an independent second rater means scores may be unconsciously influenced by familiarity with the system's intended behaviour
- **Single hardware platform** — evaluation on Apple M1 may not generalise to Windows/Linux consumer hardware, particularly regarding local LLM inference speed and quantisation behaviour

5.8 Rubric Gaps Summary

The following evaluation artefacts are not yet completed and must not be reported as finished results:

Table 13: Outstanding evaluation artefacts

Gap	Required action
Wireshark privacy audit	Capture and analyse 10-query session
TA-Lib cross-validation	10 tickers × 3 periods; 4 d.p. comparison
Formal n=20 rubric study	Expand with independent scorer
Rule-based faithfulness automation	Build numeric cross-check script
User study (n≥5, SUS)	Recruit and conduct

Gap	Required action
Backtesting / Sharpe ratio	Explicitly out of scope for current claims; deferred to future portfolio work

Chapter 6: Conclusion

6.1 Summary

PrivateFin addresses the Transparency–Privacy Gap in retail financial AI by combining deterministic technical indicator computation (RSI, MACD) with locally served Llama 3 inference via Ollama. The system delivers structured OIIR advisory narratives through a Streamlit web interface accessible to non-technical users, with educational disclaimers and graceful degradation when the local model is unavailable. The implementation realises a four-layer pipeline — data ingestion, indicator engine, OIIR prompt builder, and local LLM — with eighteen unit tests validating core deterministic behaviour.

6.2 Objectives Scorecard

Table 14: Objectives scorecard at draft-report stage

Objective	Status
Privacy (zero LLM data egress)	Architecturally achieved; Wireshark empirical audit pending (informal spot-check completed)
Deterministic indicator correctness	Partially demonstrated — pytest passes; TA-Lib validation pending
Explainability (Art. 13 benchmark)	Promising — rubric mean 7.33/8 (n=3); formal n=20 study pending
User-facing advisor bot	Achieved — Streamlit UI with 108 companies, metrics, chart, and OIIR output
Non-technical accessibility	Achieved — plain-English prompts; no code required

6.3 Originality

PrivateFin contributes a novel combination for retail-facing financial advisory: fully local LLM inference, deterministic quantitative grounding, structured OIIR explainability, and Data Protection by Design — integrated in a single system deployable on consumer hardware without cloud subscription. While individual components (yfinance, RSI, MACD, Llama 3, Streamlit, Chain-of-Thought prompting) are established technologies, their integration towards regulation-informed local advisory for novice investors addresses a gap not filled by BloombergGPT (proprietary, cloud-dependent, no structured reasoning), FinGPT (cloud-reliant at inference, sentiment-ungrounded), or FinBERT (classification-only, non-generative). The project’s originality lies in the architectural synthesis and its

treatment of EU AI Act transparency standards and GDPR data-protection principles as first-class design constraints rather than post-hoc compliance checks.

This claim is falsifiable, not asserted by fiat: it would not hold if a system already combined local-only LLM inference, deterministic indicator grounding, and OIIR-style explainability for retail equity advisory — no such system was identified among the comparators reviewed in Chapter 2, each of which fails at least one of these three properties. It would also weaken if the risk-profile and tone-modulation logic (3.4, 5.6) remains as thin as it currently is at final submission, since regulation-informed advisory needs more than an unmodified label passed to the LLM.

Measured against the brief's emphasis on significant development across all components and an ambitious algorithmic core, the current analytical engine remains two closed-form indicators rather than a learned or evolved strategy. Two routes exist for strengthening this before final submission: substantiating the privacy and transparency claims empirically (5.3, 5.8) — including the automated faithfulness checker, which is itself a non-trivial verification component that parses generated text and validates numeric and threshold claims against the deterministic layer — or extending the analytical core as discussed in 2.6. The former is prioritised, for the reasons given in 5.6.

6.4 Future Work

- Execute Wireshark privacy audit and TA-Lib cross-validation
- Expand rubric evaluation to $n=20$ with independent scorer and faithfulness automation
- Conduct user study ($n \geq 5$) with SUS and Likert instruments
- Add threshold boundary annotations to prompts; display prompt in UI
- Set explicit LLM temperature; implement @st.cache_data caching
- Extend indicators (Bollinger Bands, Stochastic RSI); add minimum date range validator; correct the flat-series RSI edge case (4.4)
- Implement SQLite audit log aligned with the Article 11 / Annex IV documentation benchmark
- Remove or integrate unused dependencies (langchain, langgraph, scikit-learn)
- Consider backtesting only if discrete trading signals are introduced as a separate module

6.5 Closing Reflection

At draft-report stage, PrivateFin is a functional, modular prototype with encouraging preliminary explainability scores (rubric mean 7.33/8, treated as indicative rather than conclusive at $n=3$) and solid unit-test coverage of its deterministic core. The principal remaining work is empirical: verifying privacy claims through Wireshark, validating indicators against TA-Lib, scaling explanation evaluation to $n=20$ with independent scoring, and gathering user trust data through the planned study.

More broadly, PrivateFin’s central bet — that explainability and privacy are worth pursuing even against the higher predictive accuracy a learned-strategy system might achieve — reflects a live tension in financial AI: the most capable systems are often the least auditable, and this project argues one concrete position, not a universal answer, on that trade-off.

References

- Appel, G. (1979) *The Moving Average Convergence-Divergence Method*. Signalert Corporation.
- Araci, D. (2019) ‘FinBERT: Financial Sentiment Analysis with Pre-trained Language Models’, *arXiv preprint* arXiv:1908.10063.
- Aroussi, R. (2019) *yfinance*. Available at: <https://pypi.org/project/yfinance/> (Accessed: 10 July 2026).
- Bender, E.M., Gebru, T., McMillan-Major, A. and Shmitchell, S. (2021) ‘On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?’, *Proceedings of FAccT 2021*, pp. 610–623.
- Brooke, J. (1996) ‘SUS: A “quick and dirty” usability scale’, in Jordan, P.W. et al. (eds.) *Usability Evaluation in Industry*. London: Taylor & Francis, pp. 189–194.
- Devlin, J. et al. (2019) ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding’, *NAACL-HLT 2019*, pp. 4171–4186.
- European Union (2016) Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*, L 119. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Accessed: 10 July 2026).
- European Union (2024) Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*, L series. Available at: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj> (Accessed: 10 July 2026).
- Grattafiori, A. et al. (2024) ‘The Llama 3 Herd of Models’, *arXiv preprint* arXiv:2407.21783.
- Hu, E.J. et al. (2022) ‘LoRA: Low-Rank Adaptation of Large Language Models’, *Proceedings of ICLR 2022* (arXiv:2106.09685).
- López de Prado, M. (2018) *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.
- Malo, P. et al. (2014) ‘Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts’, *Journal of the Association for Information Science and Technology*, 65(4), pp. 782–796.
- Manning, C.D., Raghavan, P. and Schütze, H. (2008) *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.

McKinney, W. (2010) 'Data Structures for Statistical Computing in Python', *Proceedings of SciPy 2010*, pp. 56–61.

Ollama (2023) *Ollama*. Available at: <https://ollama.com> (Accessed: 10 July 2026).

Rudin, C. (2019) 'Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead', *Nature Machine Intelligence*, 1(5), pp. 206–215.

Streamlit (2019) *Streamlit*. Available at: <https://streamlit.io> (Accessed: 10 July 2026).

Wei, J. et al. (2022) 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models', *arXiv preprint arXiv:2201.11903*.

Wilder, J.W. (1978) *New Concepts in Technical Trading Systems*. Greensboro, NC: Trend Research.

Wu, S. et al. (2023) 'BloombergGPT: A Large Language Model for Finance', *arXiv preprint arXiv:2303.17564*.

Yang, H. et al. (2023) 'FinGPT: Open-Source Financial Large Language Models', *arXiv preprint arXiv:2306.06031*.